DLDCA Lab 2: Verilog

Week 3

August 13, 2026

Introduction

In this lab we are going to start with a simple set of problems through which we will understand the basics of verilog syntax and semantics. The lab consists of three tasks each with the goal of teaching combinational circuits, along with an additional graded task.

Necessary tools

We shall be using `icarus-verilog` (or) `iverilog` in order to compile verilog HDL (Hardware Description Language). To visualize waveforms we shall use the VS Code extension 'Vaporview'. Before starting the lab, ensure that both tools are installed in your system. This can be verified by running the following:

```
$ iverilog
iverilog: no source files.

Usage: iverilog [-EiRSuvV] [-B base] [-c cmdfile|-f cmdfile]
                [-g1995|-g2001|-g2005|-g2005-sv|-g2009|-g2012] [-g<feature>]
                [-D macro[=defn]] [-I includedir] [-L moduledir]
                [-M [mode=]depfile] [-m module]
                [-N file] [-o filename] [-p flag=value]
                [-s topmodule] [-t target] [-T min|typ|max]
                [-W class] [-y dir] [-Y suf] [-l file] source_file(s)

See the man page for details.
```

Structure of submission/templates

Submission format:

```
your-roll-no/
|- labexam/
    |- universal_shift_reg.v (TODO)
    |- tb_labexam.v
|- task1/
    |- flipflops.v (TODO)
    |- tb_task1.v
|- task2/
    |- buggy_module.v (TODO)
    |- tb_task2.v
    |- bugs.txt       (TODO)
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz`. The command given below can be used to do the same

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Note for Mac users: The new makefile ensures the `-o` flag appears before the list of files. If make fails to re-build try `make -B`.

Tasks

Lab Exam: Universal Shift Register

So far, you have built sequential circuits that do one thing: a flip-flop that holds a value, or a shift register that always shifts in one direction. Real registers are rarely this limited — a register may need to *hold*, *load* a new value, or *shift* left or right, all depending on a control signal.

In this task, you will build a **Universal Shift Register**: a single 4-bit register that can perform four different operations, selected by a 2-bit `mode` input, on every rising edge of the clock.

The four modes are as follows:

mode	Operation
00	Hold (register keeps its current value)
01	Shift right (new bit enters from the MSB side)
10	Shift left (new bit enters from the LSB side)
11	Parallel load (register takes the value of `parallel_in`)

Note that unlike the shift registers you built previously, this circuit has only a *single* serial input line, `serial_in`, reused for both shift-left and shift-right — which side it enters from depends entirely on `mode`.

An example of the expected behaviour is shown below. The register starts at `0000` and is reset at cycle 0.

Cycle	mode	serial_in	parallel_in	Operation	q (after edge)
1	11	–	1011	Load	1011
2	00	–	–	Hold	1011
3	10	1	–	Shift left	0111
4	10	0	–	Shift left	1110
5	01	1	–	Shift right	1111

Refer to the given template code and fill it in to replicate this logic. Make sure to test your implementation with the testbench provided.

Compilation instructions:

```
iverilog -o testbench testbench.v module.v
vvp testbench
```

Hints

- Since the register must choose between four operations based on `mode`, use a `case` statement instead of a chain of `if-else`:

  ```
  case (select_signal)
      value1: begin
          // statements executed if select_signal == value1
      end
      default: begin
          // statements executed if select_signal matches nothing above
      end
  endcase
  ```

  Always include a `default` case — this prevents Verilog from accidentally inferring an unwanted latch.

- You can reuse your shift register logic from the previous lab for the shift-left/shift-right cases, but note that the incoming bit now enters from either side depending on `mode`.
- Handle the synchronous reset case as well. Update the values rising clock edge only.
- Think carefully about which bit gets dropped and which side the new bit enters from. See the testcases given above.

Task 1. Wires vs. Registers

So far, you may have come across the terms `wire` and `reg` in Verilog. The distinction is important: a `wire` simply reflects whatever value is driven onto it, while a `reg` holds a value in storage until it's explicitly updated. Both are essential—you'll use `wire` for combinational logic driven by `assign` statements, and `reg` for state stored inside procedural blocks like `always`.

To make this concrete, we will build three components:

- A **D Flip-Flop**, which updates its output only on a clock edge (rising or falling).
- A **T Flip-Flop**, which flips its output when the clock edge is rising and the toggle (`t`) is set.
- A **buffer**, which simply replicates its input to the output without any storage.

A quick note on procedural blocks: they execute their statements sequentially, but only when a signal in their sensitivity list changes. For example:

```
always @(posedge clk) begin
...  // this code is executed line by line every time
// the value of 'clk' changes from 0 to 1
end
```

Refer to the given template code and fill it in to replicate the logic described below for each component. Make sure to test your implementation with the accompanying testbench.

Part A: D Flip-Flop

Build a D Flip-Flop: on a rising (or falling, as specified in the template) clock edge, the output register takes on the value currently present on the data input `d`. Outside of the clock edge, the output must hold its previous value, regardless of any change on `d`.

Part B: T Flip-Flop

Build a T Flip-Flop: on a rising clock edge, if the toggle input `t` is `1`, the output register flips its value (if it was `1` it becomes `0`, and vice versa). If `t` is `0` on the rising edge, the output holds its previous value. As with the D Flip-Flop, the output must not change at any time other than the clock edge.

Part C: Buffer

Build a buffer: the output is simply a continuous reflection of the input, with no storage or dependence on a clock. Output must change immediately with input, using a `wire` and `assign` statement.

To run your code, use the makefile:

```
$ make task1
```

Task 2: Shift register debugging

For this task, you have been provided with various implementations of a 4-bit shift register, each of which does not function as expected due to intentional bugs.

A 4-bit shift register in our implementation is a device which takes a 1-bit input, a clock input, and gives a 4-bit output. Every cycle, it shifts its register value by one bit to the left and inserts the input in the least significant bit (LSB).

Note: Think about how you can use a 4-bit register to build a 8, 16 and 32-bit registers!

An example of the expected behaviour of the shift register is shown below:

Clock Cycle	Input	Output
0	0	0000
1	1	0001
2	1	0011
3	0	0110
4	1	1101
5	0	1010

You are expected to use compile errors and gtkwave outputs to debug, find and fix these errors. There are ≥ 1 errors per wrong implementation. These may include:

- Syntax errors (e.g., undeclared variables, wrong assignments)
- Incorrect shifting logic (e.g., wrong direction or wrong bit concatenation)
- Misuse of non-blocking vs blocking assignments
- Missing or incorrect sensitivity list
- Potential issues with chaining registers

Apart from submitting a fixed implementation of the same, write all the bugs you find in `bugs.txt`, in the following format. List every bug separately, even if two bugs are on the same line or in the same module. An example is (use line numbers in original file):

```
[shiftReg2] line 14:  assign answer = staten_island is old -- changed to always
block
```

You have been provided a template module file (`buggy_module.v`) and a testbench (`tb_task2.v`).

To run your code, use:

```
$ make task2
```